LAPORAN PRATIKUM STRUKTUR DATA

"SINGLE LINKED LIST"

OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : Dr. WAHYUDI, S.T, M.T
ASISTEN PRAKTIKUM : M. GALID A.

FAKULTAS TEKNOLOGI INFORMASI DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum sekaligus sebagai bentuk penerapan materi yang telah dipelajari.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep struktur data Single Linked List serta penerapannya dalam pemrograman Java. Praktikum ini memberikan gambaran mengenai bagaimana teori yang telah dipelajari dapat diimplementasikan secara langsung dalam bentuk program sederhana.

Penulis menyadari bahwa laporan ini masih belum sempurna. Oleh karena itu, penulis mengharapkan saran yang dapat membantu dalam penyempurnaan laporan di masa mendatang.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
BAB II PEMBAHASAN 2
2.1 Dasar Teori 2
2.2 Penjelasan Kode Program 2
2.2.1 Class NodeSLL_2511531007 2
2.2.2 Class TambahSLL_2511531007 3
2.2.3 Class PencarianSLL_2511531007 5
2.2.4 Class HapusSLL_2511531007 6
2.3 Tugas Praktikum 8
2.3.1 Class Pasien_2511531007 8
2.3.2 Class RumahSakit_2511531007 9
BAB III PENUTUP 13
3.1 Kesimpulan 13
3.2 Saran 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengolahan data yang terstruktur dan efisien menjadi hal yang penting, terutama ketika data yang digunakan dapat berubah-ubah jumlahnya. Salah satu permasalahan yang sering muncul adalah keterbatasan penggunaan array yang memiliki ukuran tetap serta membutuhkan penggeseran elemen saat terjadi penambahan atau penghapusan data.

Untuk mengatasi hal tersebut, digunakan struktur data *linked list* yang bersifat dinamis. *Linked list* terdiri dari node-node yang saling terhubung melalui pointer, sehingga penambahan dan penghapusan data dapat dilakukan dengan lebih fleksibel tanpa harus menggeser elemen lain. Hal ini membuat *linked list* lebih efisien dalam kondisi tertentu dibandingkan array.

Dalam praktikum ini dipelajari implementasi *singly linked list* beserta operasi dasarnya, seperti penambahan node, penghapusan node, pencarian data, dan traversal. Selain itu, konsep *linked list* juga diterapkan pada kasus antrian pasien rumah sakit yang menggunakan prinsip FIFO (*First In First Out*), di mana data yang masuk lebih dulu akan diproses lebih dahulu.

Melalui praktikum ini, diharapkan dapat memahami konsep dasar *linked list* serta cara penerapannya dalam menyelesaikan permasalahan pengelolaan data yang bersifat dinamis.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dasar struktur data *singly linked list* beserta cara kerjanya.
2. Mampu mengimplementasikan operasi dasar *linked list* seperti penambahan, penghapusan, pencarian.

1.3 Manfaat

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Menambah pemahaman tentang penggunaan *linked list* sebagai alternatif struktur data yang lebih fleksibel dibanding array.
2. Melatih kemampuan dalam mengelola data secara dinamis melalui operasi penambahan, penghapusan, dan pencarian.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Praktikum ini membahas tentang struktur data *singly linked list*, yaitu salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada *linked list* memiliki dua bagian utama, yaitu data dan pointer yang menunjuk ke node berikutnya. Berbeda dengan array yang memiliki ukuran tetap dan penyimpanan berurutan di memori, *linked list* bersifat dinamis karena ukuran dapat bertambah atau berkurang sesuai kebutuhan.

Pada *singly linked list*, hubungan antar node bersifat satu arah, sehingga proses penelusuran hanya dapat dilakukan dari node pertama (*head*) menuju node terakhir. Beberapa operasi dasar yang dapat dilakukan pada *linked list* antara lain penambahan node di depan (*insert at front*), penambahan di belakang (*insert at end*), penyisipan di posisi tertentu, penghapusan node (*deletion*), serta pencarian data (*search*). Setiap operasi dilakukan dengan memanipulasi pointer antar node.

Konsep traversal juga menjadi bagian penting dalam *linked list*, yaitu proses menelusuri seluruh node dari awal hingga akhir untuk menampilkan atau mencari data. Karena tidak memiliki indeks seperti array, setiap pencarian atau akses data pada linked list harus dilakukan secara berurutan.

Dalam praktikum ini, *linked list* juga diterapkan pada kasus nyata yaitu sistem antrian pasien rumah sakit. Sistem ini menggunakan konsep FIFO (*First In First Out*), di mana data yang pertama masuk akan diproses lebih dahulu. Implementasi ini menunjukkan bahwa *linked list* dapat digunakan untuk mengelola data yang bersifat dinamis dan berurutan, seperti pada antrian.

2.2 Penjelasan Kode Program

2.2.1 Class NodeSLL_2511531007

```java
package pekan5_2511531007;

public class NodeSLL_2511531007 {
    // node bagian data
    int data_1007;
    // pointer ke node berikutnya
    NodeSLL_2511531007 next_1007;
    // konstruktor menginisialisasi node dengan data
    public NodeSLL_2511531007(int data_1007) {
        this.data_1007 = data_1007;
        this.next_1007 = null;
    }

}
```

Program ini bikin satu bagian kecil dari struktur data *linked list*, tepatnya node (simpul) untuk *singly linked list*.

Di dalam class ini ada dua komponen utama. Yang pertama data_1007, ini buat nyimpen nilai dari node tersebut, tipe datanya int. Yang kedua next_1007, ini semacam penunjuk (pointer) ke node berikutnya.

Terus ada *constructor* NodeSLL_2511531007(int data_1007). Fungsinya buat bikin node baru sekaligus langsung ngisi datanya. Jadi pas object node dibuat, nilai data_1007 langsung diisi sesuai parameter yang dikasih. Sementara itu next_1007 di-set ke null, artinya node ini belum terhubung ke node lain.

2.2.2 Class TambahSLL_2511531007

```java
package pekan5_2511531007;

public class TambahSLL_2511531007 {
    public static NodeSLL_2511531007 insertAtFront_1007(NodeSLL_2511531007 head_1007, int value_1007) {
        NodeSLL_2511531007 new_node_1007 = new NodeSLL_2511531007(value_1007);
        new_node_1007.next_1007 = head_1007;
        return new_node_1007;
    }
    // fungsi menambahkan node di akhir SLL
        public static NodeSLL_2511531007 insertAtEnd_1007(NodeSLL_2511531007 head_1007, int value_1007) {
            // buat sebuah node dengan sebuah nilai
            NodeSLL_2511531007 newNode_1007 = new NodeSLL_2511531007(value_1007);
            // jika list kosong maka node jadi head
            if (head_1007 == null) {
                return newNode_1007;
            }
        // simpan head ke variabel sementara
            NodeSLL_2511531007 last_1007 = head_1007;
            // telusuri ke node akhir
            while (last_1007.next_1007 != null) {
                last_1007 = last_1007.next_1007;
            }
            // ubah pointer
            last_1007.next_1007 = newNode_1007;
            return head_1007;
        }
        static NodeSLL_2511531007 GetNode_1007(int data_1007) {
            return new NodeSLL_2511531007(data_1007);
        }
        static NodeSLL_2511531007 insertPos_1007(NodeSLL_2511531007 headNode_1007, int position_1007, int value_1007) {
            NodeSLL_2511531007 head_1007 = headNode_1007;
            if (position_1007 < 1) {
                System.out.print("Invalid position");
            }
            if (position_1007 == 1) {
                NodeSLL_2511531007 new_node_1007 = new NodeSLL_2511531007(value_1007);
                new_node_1007.next_1007 = head_1007;
                return new_node_1007;
            } else {
                while (position_1007-- != 0) {
                    if (position_1007 == 1) {
                        NodeSLL_2511531007 newNode_1007 = GetNode_1007(value_1007);
                        newNode_1007.next_1007 = headNode_1007.next_1007;
                        headNode_1007.next_1007 = newNode_1007;
                        break;
                    }
                    headNode_1007 = headNode_1007.next_1007;
                }
            if (position_1007 != 1)
            System.out.print("Posisi di luar jangkauan");
            }
            return head_1007;
            }
```

```java
        public static void printList_1007(NodeSLL_2511531007 head_1007) {
            NodeSLL_2511531007 curr_1007 = head_1007;
                while (curr_1007.next_1007 != null) {
                    System.out.print(curr_1007.data_1007 + "-->");
                    curr_1007 = curr_1007.next_1007;
            }
            if (curr_1007.next_1007 == null) {
                System.out.print(curr_1007.data_1007);
            }
            System.out.println();
        }

        public static void main (String[] args) {
            // buat linked list 2->3->5->6
            NodeSLL_2511531007 head_1007 = new NodeSLL_2511531007(2);
            head_1007.next_1007 = new NodeSLL_2511531007(3);
            head_1007.next_1007.next_1007 = new NodeSLL_2511531007(5);
            head_1007.next_1007.next_1007.next_1007 = new NodeSLL_2511531007(6);

            // cetak list asli
            System.out.print("Senarai berantai awal:");
            printList_1007(head_1007);

            // tambahkan node baru di depan
            System.out.print("tambah 1 simpul di depan: ");
            int data_1007 = 1;
            head_1007 = insertAtFront_1007(head_1007, data_1007);

            // cetak update list
            printList_1007(head_1007);

            // tambahkan node baru di belakang
            System.out.print("tambah 1 simpul di belakang: ");
            int data2_1007 = 7;
            head_1007 = insertAtEnd_1007(head_1007, data2_1007);

            // cetak update list
            printList_1007(head_1007);
            System.out.print("tambah 1 simpul ke data 4: ");
            int data3_1007 = 4;
            int pos_1007 = 4;
            head_1007 = insertPos_1007(head_1007, pos_1007, data3_1007);

            // cetak update list
            printList_1007(head_1007);

        }


}
```

```
<terminated> TambahSLL_2511531007 [Java Application] C:\Pr
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Program ini digunakan untuk menambahkan data pada struktur data *singly linked list* dengan beberapa cara, yaitu menambah data di depan, di belakang, dan di posisi tertentu. Jadi class ini berfungsi buat mengatur bagaimana node-node di dalam linked list bisa bertambah dan tetap saling terhubung dengan benar. Linked list sendiri bekerja dengan cara setiap node menyimpan data dan alamat ke node berikutnya, jadi proses penambahannya dilakukan dengan mengatur pointer antar node.

Di dalam class ini ada beberapa method utama. Method insertAtFront_1007() dipakai buat menambahkan node di bagian paling depan linked list. Caranya dengan membuat node baru, lalu pointer next_1007 dari node baru diarahkan ke head lama. Setelah itu node baru dikembalikan sebagai head yang baru. Dengan cara ini, data baru langsung jadi elemen pertama dalam linked list.

Method insertAtEnd_1007() digunakan buat menambahkan node di bagian akhir linked list. Pertama program membuat node baru terlebih dahulu. Kalau linked list masih kosong, node baru langsung dijadikan head. Tapi kalau linked list sudah ada isinya, program akan menelusuri node satu per satu sampai menemukan node terakhir,

yaitu node yang pointer next_1007-nya bernilai null. Setelah ketemu, pointer node terakhir diubah supaya menunjuk ke node baru.

Ada juga method GetNode_1007() yang fungsinya lebih sederhana, yaitu hanya untuk membuat node baru dengan nilai tertentu. Method ini dipakai sebagai bantuan di bagian lain program supaya pembuatan node bisa dilakukan lebih ringkas.

Method insertPos_1007() digunakan untuk menambahkan node pada posisi tertentu di linked list. Program akan mengecek dulu apakah posisi yang dimasukkan valid atau tidak. Kalau posisi sama dengan 1, maka prosesnya sama seperti menambah di depan. Kalau posisinya lebih dari 1, program akan menelusuri linked list sampai posisi yang diinginkan, lalu menyisipkan node baru di antara node-node yang sudah ada. Kalau posisi di luar jangkauan, program akan menampilkan pesan error.

Method printList_1007() dipakai buat menampilkan seluruh isi linked list. Program akan menelusuri node mulai dari head sampai node terakhir, lalu mencetak datanya satu per satu dengan tanda panah (-->) sebagai penghubung antar node. Ini berguna buat melihat apakah proses penambahan data berhasil atau tidak.

Di method main(), program mulai dengan membuat linked list awal berisi data 2 -> 3 -> 5 -> 6. Setelah itu program menampilkan list awal, lalu melakukan beberapa operasi penambahan data. Pertama menambahkan angka 1 di depan, lalu menambahkan angka 7 di belakang, dan terakhir menambahkan angka 4 di posisi tertentu. Setiap selesai menambah data, linked list langsung ditampilkan lagi supaya perubahan susunannya bisa dilihat.

2.2.3 Class PencarianSLL_2511531007

```java
package pekan5_2511531007;

public class PencarianSLL_2511531007 {
    static boolean searchKey_1007(NodeSLL_2511531007 head_1007, int key_1007) {
        NodeSLL_2511531007 curr_1007 = head_1007;
        while (curr_1007 != null) {
            if (curr_1007.data_1007 == key_1007)
                return true;
            curr_1007 = curr_1007.next_1007;
        }
        return false;
    }

    public static void traversal_1007(NodeSLL_2511531007 head_1007) {
        // mulai dari head
        NodeSLL_2511531007 curr_1007 = head_1007;
        while (curr_1007 != null) {
            System.out.print(" " + curr_1007.data_1007);
            curr_1007 = curr_1007.next_1007;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511531007 head_1007 = new NodeSLL_2511531007(14);
        head_1007.next_1007 = new NodeSLL_2511531007(21);
        head_1007.next_1007.next_1007 = new NodeSLL_2511531007(13);
        head_1007.next_1007.next_1007.next_1007 = new NodeSLL_2511531007(30);
        head_1007.next_1007.next_1007.next_1007.next_1007 = new NodeSLL_2511531007(10);
        System.out.print("Penelusuran SLL : ");
        traversal_1007(head_1007);
        // data yang akan dicari
        int key_1007 = 30;
        System.out.print("cari data " +key_1007+ " = ");
        if (searchKey_1007(head_1007, key_1007))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }

}
```

```
Penelusuran SLL :  14 21 13 30 10
cari data 30 = ketemu
```

Program ini digunakan untuk melakukan pencarian data *pada singly linked list*, sekaligus menampilkan isi list tersebut.

Method searchKey_1007() dipakai untuk mencari apakah suatu nilai (key) ada di dalam linked list atau tidak. Program akan mulai dari head, lalu menelusuri node satu per satu menggunakan variabel curr_1007. Setiap node dicek apakah datanya sama dengan nilai yang dicari. Kalau ketemu, method langsung mengembalikan *true*. Kalau sampai akhir list gak ketemu juga, maka method akan mengembalikan *false.*

Method traversal_1007() digunakan untuk menampilkan seluruh isi linked list. Program mulai dari head, lalu berjalan ke node berikutnya sampai null. Setiap data dicetak ke layar dengan spasi sebagai pemisah.

Pada method main(), program membuat linked list dengan isi 14 -> 21 -> 13 -> 30 -> 10. Setelah itu, list ditampilkan menggunakan method traversal. Kemudian program menentukan data yang ingin dicari, yaitu 30. Lalu method searchKey_1007() dipanggil untuk mengecek apakah nilai tersebut ada di dalam list.

Jika data ditemukan, program akan menampilkan tulisan “ketemu”. Kalau tidak ditemukan, akan muncul “tidak ada”.

2.2.4 Class HapusSLL_2511531007

```java
package pekan5_2511531007;

public class HapusSLL_2511531007 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511531007 deleteHead_1007(NodeSLL_2511531007 head_1007) {
        // jika SLL kosong
        if (head_1007 == null)
            return null;
        // pindahkan head ke node berikutnya
        head_1007 = head_1007.next_1007;
        // return head baru
        return head_1007;
    }
    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511531007 removeLastNode_1007(NodeSLL_2511531007 head_1007) {
        // jika list kosong, return null
        if (head_1007 == null) {
            return null;
        }
        // jika list satu node, hapus node dan return null
        if (head_1007.next_1007 == null) {
            return null;
        }
        // temukan node terakhir ke dua
        NodeSLL_2511531007 secondLast_1007 = head_1007;
        while (secondLast_1007.next_1007.next_1007 != null) {
            secondLast_1007 = secondLast_1007.next_1007;
        }
        // hapus node terakhir
        secondLast_1007.next_1007 = null;
        return head_1007;
    }
    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511531007 deleteNode_1007(NodeSLL_2511531007 head_1007, int position_1007) {
        NodeSLL_2511531007 temp_1007 = head_1007;
        NodeSLL_2511531007 prev_1007 = null;
        // jika linked list null
        if (temp_1007 == null)
            return head_1007;
        // kasus 1: head dihapus
        if (position_1007 == 1) {
            head_1007 = temp_1007.next_1007;
            return head_1007;
        }
```

```java
        // kasus 2: menghapus node di tengah
        // telusuri ke node yang di hapus
    for (int i_1007 = 1; temp_1007 != null && i_1007 < position_1007; i_1007++) {
        prev_1007 = temp_1007;
        temp_1007 = temp_1007.next_1007;
    }
    // jika ditemukan, hapus node
    if (temp_1007 != null) {
        prev_1007.next_1007 = temp_1007.next_1007;
    } else {
        System.out.println("Data tidak ada");
    }
        return head_1007;
    }
    // fungsi mencetak SLL
    public static void printList_1007(NodeSLL_2511531007 head_1007) {
        NodeSLL_2511531007 curr_1007 = head_1007;
        while (curr_1007.next_1007 != null) {
            System.out.print(curr_1007.data_1007+"-->");
            curr_1007 = curr_1007.next_1007;
        }
        if (curr_1007.next_1007 == null) {
            System.out.print(curr_1007.data_1007);
        }
        System.out.println();

    }
    // kelas main
    public static void main(String[] args) {
        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
        NodeSLL_2511531007 head_1007 = new NodeSLL_2511531007(1);
        head_1007.next_1007 = new NodeSLL_2511531007(2);
        head_1007.next_1007.next_1007 = new NodeSLL_2511531007(3);
        head_1007.next_1007.next_1007.next_1007 = new NodeSLL_2511531007(4);
        head_1007.next_1007.next_1007.next_1007.next_1007 = new NodeSLL_2511531007(5);
        head_1007.next_1007.next_1007.next_1007.next_1007.next_1007 = new NodeSLL_2511531007(6);
        // cetak list awal
        System.out.println("list awal: ");
        printList_1007(head_1007);

         //hapus head
         head_1007 = deleteHead_1007(head_1007);
         System.out.println("List setelah head dihapus: ");
         printList_1007(head_1007);

         //hapus node terakhir
         head_1007 = removeLastNode_1007(head_1007);
         System.out.println("List setelah simpul terakhir di hapus: ");
         printList_1007(head_1007);

         // deleting node at position 2
         int position_1007 = 2;
         head_1007 = deleteNode_1007(head_1007, position_1007);

         // print list after deletion
         System.out.println("List setelah posisi 2 dihapus: ");
         printList_1007(head_1007);
    }
}
```

```
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Program ini digunakan untuk menghapus node pada *singly linked list*. Di dalam class ini ada beberapa method penting untuk proses penghapusan:

Method deleteHead_1007() dipakai untuk menghapus node paling depan (head). Kalau linked list kosong (head_1007 == null), langsung dikembalikan null. Tapi kalau ada isi, head dipindahkan ke node berikutnya (head_1007.next_1007). Jadi node pertama otomatis “hilang” dari list karena sudah tidak ada yang menunjuk ke dia.

Method removeLastNode_1007() digunakan untuk menghapus node paling akhir. Kalau list kosong, gak ada yang dihapus. Kalau cuma ada satu node, setelah dihapus hasilnya jadi null. Kalau lebih dari satu, program akan mencari node kedua dari belakang (second last). Caranya dengan menelusuri sampai next.next == null. Setelah ketemu, pointer next dari node tersebut di-set ke null, jadi node terakhir terputus dari list.

Method deleteNode_1007() digunakan untuk menghapus node pada posisi tertentu. Program mulai dari head dan menelusuri node satu per satu sampai posisi yang diminta. Ada dua kondisi utama:

- Kalau posisi 1, berarti sama seperti hapus head.
- Kalau posisi di tengah, program menyimpan node sebelumnya (prev_1007) dan node yang akan dihapus (temp_1007). Setelah ketemu, pointer dari node sebelumnya diarahkan ke node setelahnya (prev.next = temp.next). Jadi node di tengah "dilompati" dan otomatis hilang dari rantai. Kalau posisi yang diminta tidak ada (kelebihan panjang list), program akan menampilkan pesan "Data tidak ada".

Method printList_1007() berfungsi untuk menampilkan isi linked list. Sama seperti sebelumnya, dia berjalan dari head sampai node terakhir sambil mencetak data dengan format -->. Node terakhir dicetak tanpa panah.

Pada method main(), program membuat linked list awal: 1 -> 2 -> 3 -> 4 -> 5 -> 6

Lalu dilakukan beberapa operasi:

- Pertama, list awal ditampilkan.
- Kedua, head dihapus, jadi list berubah jadi 2 -> 3 -> 4 -> 5 -> 6.
- Ketiga, node terakhir dihapus, jadi 2 -> 3 -> 4 -> 5.
- Terakhir, node pada posisi ke-2 dihapus, jadi 2 -> 4 -> 5.

2.3 Tugas Praktikum

2.3.1 Class Pasien_2511531007

```java
package tugasPraktikum_pekan5_2511531007;

public class Pasien_2511531007 {
    String namaPasien_1007;
    String penyakit_1007;
    int noAntrian_1007;
    Pasien_2511531007 next_1007;

    // Constructor
    public Pasien_2511531007(String namaPasien_1007, String penyakit_1007, int noAntrian_1007) {
        this.namaPasien_1007 = namaPasien_1007;
        this.penyakit_1007 = penyakit_1007;
        this.noAntrian_1007 = noAntrian_1007;
        this.next_1007 = null;
    }

    // Getter
    public String getNamaPasien_1007() {return namaPasien_1007;}
    public String getPenyakit_1007() {return penyakit_1007;}
    public int getNoAntrian_1007() {return noAntrian_1007;}
    public Pasien_2511531007 getNext_1007() {return next_1007;}

    // Setter
    public void setNamaPasien_1007(String namaPasien_1007) {this.namaPasien_1007 = namaPasien_1007;}
    public void setPenyakit_1007(String penyakit_1007) {this.penyakit_1007 = penyakit_1007;}
    public void setNoAntrian_1007(int noAntrian_1007) {this.noAntrian_1007 = noAntrian_1007;}
    public void setNext_1007(Pasien_2511531007 next_1007) {this.next_1007 = next_1007;}
}
```

Program ini bikin satu node untuk *linked list* antrian pasien. Setiap objek Pasien_2511531007 menyimpan data pasien berupa nama, penyakit, dan nomor antrian, serta punya pointer next_1007 yang menunjuk ke pasien berikutnya.

Constructor dipakai buat langsung mengisi data saat objek dibuat, dan next_1007 di-set null karena belum terhubung ke node lain. Getter digunakan untuk mengambil data pasien, sedangkan setter untuk mengubahnya.

2.3.2 Class RumahSakit_2511531007

```java
package tugasPraktikum_pekan5_2511531007;

import java.util.Scanner;

public class RumahSakit_2511531007 {

    // Counter statis untuk nomor antrian otomatis
    static int counter_1007 = 0;

    public static void main(String[] args) {
        Scanner scanner_1007 = new Scanner(System.in);
        Pasien_2511531007 head_1007 = null; // Head list dimulai null
        int pilihan_1007 = 0;

        do {
            // Tampilkan Menu
            System.out.println("\n=== Antrian Rumah Sakit NIM:2511531007===");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");

            if (scanner_1007.hasNextInt()) {
                pilihan_1007 = scanner_1007.nextInt();
                scanner_1007.nextLine();
            } else {
                System.out.println("Input harus berupa angka!");
                scanner_1007.nextLine();
                continue;
            }

            switch (pilihan_1007) {
                case 1:
                    System.out.print("Masukkan Nama Pasien: ");
                    String nama_1007 = scanner_1007.nextLine();
                    System.out.print("Masukkan Keluhan : ");
                    String keluhan_1007 = scanner_1007.nextLine();
                    head_1007 = daftarkanPasien_1007(head_1007, nama_1007, keluhan_1007);
                    break;

                case 2:
                    head_1007 = panggilPasien_1007(head_1007);
                    break;

                case 3:
                    tampilkanAntrian_1007(head_1007);
                    break;

                case 4:
                    System.out.print("Masukkan nama pasien yang dicari: ");
                    String kataKunci_1007 = scanner_1007.nextLine();
                    cariPasien_1007(head_1007, kataKunci_1007);
                    break;

                case 5:
                    cekStatus_1007(head_1007);
                    break;

                case 6:
                    System.out.println("Keluar dari program...");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan_1007 != 6);

        scanner_1007.close();

    }
```

```java
    // 1. Daftarkan Pasien
    public static Pasien_2511531007 daftarkanPasien_1007(Pasien_2511531007 head_1007, String nama, String keluhan) {
        // Increment counter otomatis
        counter_1007++;

        // Buat node baru
        Pasien_2511531007 nodeBaru_1007 = new Pasien_2511531007(nama, keluhan, counter_1007);

        // Jika list kosong, node baru jadi head
        if (head_1007 == null) {
            head_1007 = nodeBaru_1007;
        } else {
            // Cari node terakhir
            Pasien_2511531007 temp_1007 = head_1007;
            while (temp_1007.next_1007 != null) {
                temp_1007 = temp_1007.next_1007;
            }
            // Sambungkan node baru ke akhir
            temp_1007.next_1007 = nodeBaru_1007;
        }

        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + nodeBaru_1007.noAntrian_1007);
        return head_1007;
    }

    // 2. Panggil Pasien
    public static Pasien_2511531007 panggilPasien_1007(Pasien_2511531007 head_1007) {
        if (head_1007 == null) {
            System.out.println("Antrian kosong, tidak ada pasien untuk dipanggil.");
            return head_1007;
        }

        System.out.println("\n--- Memanggil Pasien ---");
        System.out.println("Nomor Antrian: " + head_1007.noAntrian_1007);
        System.out.println("Nama: " + head_1007.namaPasien_1007);
        System.out.println("Keluhan: " + head_1007.penyakit_1007);
        System.out.println("-----------------------\n");

        // Geser head ke node berikutnya
        head_1007 = head_1007.next_1007;
        return head_1007;
    }

    // 3. Tampilkan Antrian
    public static void tampilkanAntrian_1007(Pasien_2511531007 head_1007) {
        if (head_1007 == null) {
            System.out.println("Antrian kosong.");
            return;
        }

        System.out.println("\n=== Daftar Antrian Saat Ini ===");
        Pasien_2511531007 curr_1007 = head_1007;
        int posisi_1007 = 1;
        while (curr_1007 != null) {
            System.out.println("Posisi " + posisi_1007 + ":");
            System.out.println("  No. Antrian: " + curr_1007.noAntrian_1007);
            System.out.println("  Nama: " + curr_1007.namaPasien_1007);
            System.out.println("  Keluhan: " + curr_1007.penyakit_1007);
            curr_1007 = curr_1007.next_1007;
            posisi_1007++;
        }
        System.out.println("============================\n");
    }

    // 4. Cari Pasien
    public static void cariPasien_1007(Pasien_2511531007 head_1007, String namaCari) {
        if (head_1007 == null) {
            System.out.println("Antrian kosong.");
            return;
        }

        Pasien_2511531007 curr_1007 = head_1007;
        boolean ditemukan_1007 = false;

        while (curr_1007 != null) {
            if (curr_1007.namaPasien_1007.equalsIgnoreCase(namaCari)) {
                System.out.println("\nPasien Ditemukan!");
                System.out.println("No. Antrian: " + curr_1007.noAntrian_1007);
                System.out.println("Nama: " + curr_1007.namaPasien_1007);
                System.out.println("Keluhan: " + curr_1007.penyakit_1007);
                ditemukan_1007 = true;
                break;
            }
            curr_1007 = curr_1007.next_1007;
        }

        if (!ditemukan_1007) {
            System.out.println("Pasien dengan nama '" + namaCari + "' tidak ditemukan dalam antrian.");
        }
    }

    // 5. Cek Status Antrian
    public static void cekStatus_1007(Pasien_2511531007 head_1007) {
        System.out.println("\n=== Status Antrian ===");
        if (head_1007 == null) {
            System.out.println("Total Pasien: 0");
            System.out.println("Pasien Terdepan: Tidak ada (Antrian Kosong)");
        } else {
            // Hitung total pasien
            int total_1007 = 0;
            Pasien_2511531007 curr_1007 = head_1007;
            while (curr_1007 != null) {
                total_1007++;
                curr_1007 = curr_1007.next_1007;
            }

            System.out.println("Total Pasien: " + total_1007);
            System.out.println("Pasien Terdepan: " + head_1007.namaPasien_1007 + " (No. Antrian: " + head_1007.noAntrian_1007 + ")");
        }
        System.out.println("==================\n");
    }
}
```

```
=== Antrian Rumah Sakit NIM:2511531007===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Budi Santoso
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1
```

```
=== Antrian Rumah Sakit NIM:2511531007===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3

=== Daftar Antrian Saat Ini ===
Posisi 1:
  No. Antrian: 1
  Nama: Budi Santoso
  Keluhan: Demam
==============================
```

```
=== Antrian Rumah Sakit NIM:2511531007===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien yang dicari: Budi santoso

Pasien Ditemukan!
No. Antrian: 1
Nama: Budi Santoso
Keluhan: Demam
```

```
=== Antrian Rumah Sakit NIM:2511531007===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5

=== Status Antrian ===
Total Pasien: 1
Pasien Terdepan: Budi Santoso (No. Antrian: 1)
====================
```

```
=== Antrian Rumah Sakit NIM:2511531007===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2

--- Memanggil Pasien ---
Nomor Antrian: 1
Nama: Budi Santoso
Keluhan: Demam
------------------------
```

```
=== Antrian Rumah Sakit NIM:2511531007===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Keluar dari program...
```

Program ini digunakan untuk mengelola antrian pasien rumah sakit menggunakan *singly linked list*. Sistem antrian yang digunakan adalah FIFO (*First In First Out*), yaitu pasien yang datang lebih dulu akan dipanggil lebih dulu.

Pada bagian main(), program menampilkan menu interaktif yang bisa dipilih oleh pengguna. Menu tersebut terdiri dari pendaftaran pasien, pemanggilan pasien, menampilkan antrian, mencari pasien, mengecek status antrian, dan keluar dari program. Input dari pengguna dibaca menggunakan Scanner.

Method daftarkanPasien_1007() digunakan untuk menambahkan pasien ke dalam antrian. Setiap pasien akan mendapatkan nomor antrian otomatis dari variabel counter_1007. Jika antrian masih kosong, pasien baru akan menjadi head. Jika tidak, pasien akan ditambahkan di bagian akhir linked list.

Method panggilPasien_1007() digunakan untuk memanggil pasien paling depan. Data pasien akan ditampilkan, lalu head digeser ke node berikutnya sehingga pasien tersebut keluar dari antrian.

Method tampilkanAntrian_1007() berfungsi untuk menampilkan seluruh daftar pasien dalam antrian, mulai dari posisi pertama hingga terakhir, beserta nomor antrian, nama, dan keluhan.

Method cariPasien_1007() digunakan untuk mencari pasien berdasarkan nama. Program akan menelusuri linked list satu per satu dan mencocokkan nama pasien. Jika ditemukan, data pasien akan ditampilkan.

Method cekStatus_1007() digunakan untuk melihat kondisi antrian, yaitu jumlah total pasien dan siapa pasien yang berada di posisi paling depan.

BAB III
PENUTUP

3.1 Kesimpulan

Praktikum ini bertujuan untuk memahami dan mengimplementasikan struktur data *singly linked list* dalam pemrograman Java. Melalui beberapa percobaan yang telah dilakukan, dapat disimpulkan bahwa linked list terdiri dari node-node yang saling terhubung, di mana setiap node menyimpan data dan pointer ke node berikutnya.

Dalam praktikum ini telah berhasil dibuat berbagai operasi dasar pada *linked list*, seperti penambahan node di depan, belakang, dan posisi tertentu, pencarian data, serta penghapusan node. Selain itu, juga dilakukan penerapan linked list dalam kasus nyata berupa sistem antrian pasien rumah sakit yang menggunakan konsep FIFO (*First In First Out*).

Hasil praktikum menunjukkan bahwa *linked list* lebih fleksibel dibandingkan array karena tidak memiliki batasan ukuran tetap dan dapat bertambah atau berkurang secara dinamis. Dengan demikian, penggunaan linked list sangat cocok untuk mengelola data yang sering mengalami perubahan, seperti pada sistem antrian.

3.2 Saran

1. Mahasiswa diharapkan dapat lebih sering berlatih mengerjakan soal atau membuat program secara mandiri, sehingga saat menghadapi ujian akhir praktikum tidak mengalami kesulitan dalam memahami maupun menyelesaikan soal yang diberikan.
2. Selain itu, mahasiswa sebaiknya tidak hanya mengikuti langkah praktikum, tetapi juga mencoba memahami alur program agar dapat mengerjakan tanpa bergantung pada contoh yang sudah ada.
3. Diharapkan asisten praktikum dapat memberikan evaluasi atau umpan balik terhadap laporan maupun program yang telah dibuat, sehingga mahasiswa dapat mengetahui letak kesalahan dan melakukan perbaikan.
4. Dengan adanya latihan yang cukup dan evaluasi yang jelas, diharapkan proses pembelajaran praktikum dapat menjadi lebih efektif dan hasil yang diperoleh mahasiswa menjadi lebih maksimal.

DAFTAR PUSTAKA

[1] W. Wahyudi, *Struktur Data dengan Java*. CV Eureka Media Aksara, 2025. [Online]. Available: https://books.google.co.id/books?id=EN2lEQAAQBAJ.